Ali Moallin

Electrical and Computer Engineering

Regarding how I wrote the VHDL code for the System Controller state diagram [4], I made it so the implementation was a finite state machine with 5 states. These states being InitS, LoadS, AddS, ShiftS, and DoneS. The controller updates on the falling edge and the registers update on the rising edge.

The original controller template included a counter (CNT), I moved this counter's behavior into its own VHDL module, naming it CounterN [5]. I made the counter have ports, Clk, Clear, Enable, and C4. Clear is connected to Load, Enable is connected to Shift, and C4 is connected into the controller.

The waveform [1] shows that the multiplier works because when Start is asserted, the circuit starts the multiplication cycle, after the cycles, Done becomes high, this shows the operation is finished. Also, the product signal matches the multiplication result for corresponding Multiplier and Multiplicant values.

One modification that I think would be very convenient is a RESET, this makes it so the user does not have to wait for several cycles, but manually reset it whenever they want. This can be very useful in certain situations.

Appendix

Figure 1 *Wave Simulation*

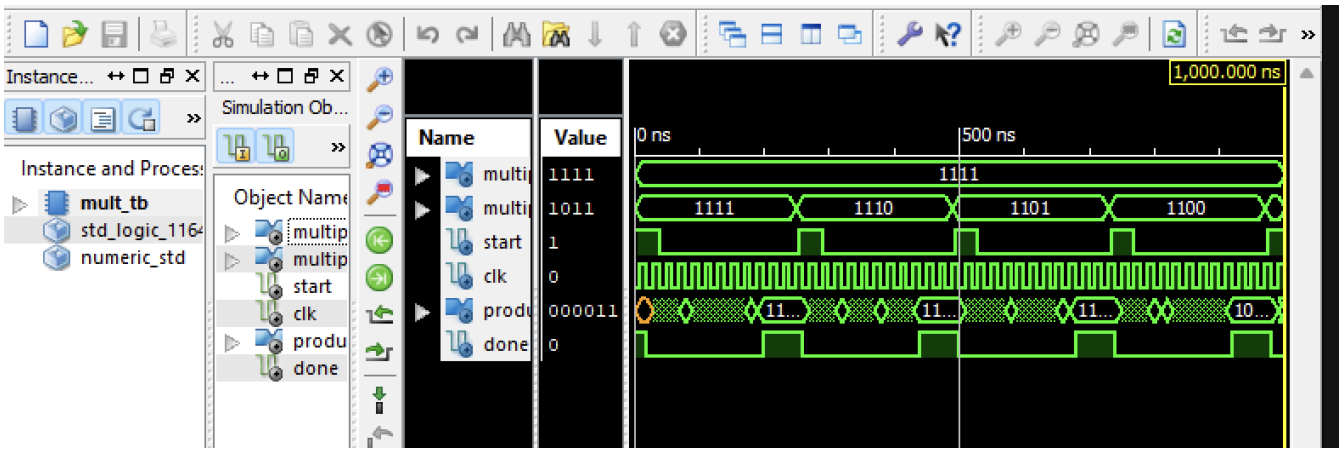

Figure 2 *Test Bench*

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY mult_tb IS
END mult_tb;

ARCHITECTURE behavior OF mult_tb IS

    COMPONENT MULTTOP
    PORT(
         Multiplier : IN  std_logic_vector(3 downto 0);
         Multiplicand : IN  std_logic_vector(3 downto 0);
         Product : OUT  std_logic_vector(7 downto 0);
         Start : IN  std_logic;
         Done : OUT  std_logic;
         Clk : IN  std_logic
        );
    END COMPONENT;

   --Inputs
   signal Multiplier : std_logic_vector(3 downto 0) := (others => '0');
   signal Multiplicand : std_logic_vector(3 downto 0) := (others => '0');
   signal Start : std_logic := '0';
   signal Clk : std_logic := '0';

   --Outputs
   signal Product : std_logic_vector(7 downto 0);
   signal Done : std_logic;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: MULTTOP PORT MAP (
          Multiplier => Multiplier,
          Multiplicand => Multiplicand,
          Product => Product,
          Start => Start,
          Done => Done,
          Clk => Clk
        );

   Clk <= not Clk after 10 ns;
```

```
   Clk <= not Clk after 10 ns;

   -- Stimulus process
   stim_proc: process
   begin
         for i in 15 downto 0 loop
            Multiplier <= std_logic_vector(to_unsigned(i,4));
            for j in 15 downto 0 loop
               Multiplicand <= std_logic_vector(to_unsigned(j,4));
               Start <= '0', '1' after 5 ns, '0' after 40 ns;
               wait for 50 ns;
               wait until Done = '1';
               assert(to_integer(UNSIGNED(Product))=(i*j))
                  report "Incorrect product" severity NOTE;
               wait for 50 ns;
            end loop;
         end loop;
      wait;
   end process;
);
```

Figure 3 *Multiplier*

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity MULTTOP is
5      Port ( Multiplier : in  STD_LOGIC_VECTOR (3 downto 0);
6             Multiplicand : in  STD_LOGIC_VECTOR (3 downto 0);
7             Product : out  STD_LOGIC_VECTOR (7 downto 0);
8             Start : in  STD_LOGIC;
9             Done : out  STD_LOGIC;
10            Clk : in  STD_LOGIC);
11 end MULTTOP;
12
13 architecture Behavioral of MULTTOP is
14    component Controller
15        generic (N: integer := 2);
16        Port ( Clk : in  STD_LOGIC;
17               Q0 : in  STD_LOGIC;
18               C4  : in STD_LOGIC;
19               Start : in  STD_LOGIC;
20               Load : out  STD_LOGIC;
21               Shift : out  STD_LOGIC;
22               AddA : out  STD_LOGIC;
23               Done : out  STD_LOGIC);
24    end component;
25
26    component AdderN
27       generic (N: integer := 4);
28       port (   A: in std_logic_vector(N-1 downto 0);
29                B: in std_logic_vector(N-1 downto 0);
30                S: out std_logic_vector(N downto 0)
31             );
32    end component;
33
34    component RegN
35    generic (N: integer := 4);
36    port (   Din: in std_logic_vector(N-1 downto 0);
37             Dout: out std_logic_vector(N-1 downto 0);
38             Clk: in std_logic;
39             Load: in std_logic;
40             Shift: in std_logic;
41             Clear: in std_logic;
42             SerIn: in std_logic
43          );
44    end component;
```

Figure 4 *Controller*

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Controller is
6      generic (N: integer := 2);
7      Port ( Clk : in  STD_LOGIC;
8             Q0 : in  STD_LOGIC;
9             C4 : in STD_LOGIC;
10            Start : in  STD_LOGIC;
11            Load : out  STD_LOGIC;
12            Shift : out  STD_LOGIC;
13            AddA : out  STD_LOGIC;
14            Done : out  STD_LOGIC);
15 end Controller;
16
17 architecture Behavioral of Controller is
18    type states is (
19
20 -- Uncomment the following library declaration if instantiating
21 -- any Xilinx primitives in this code.
22 --library UNISIM;
23 --use UNISIM.VComponents.all;
24 InitS,LoadS,AddS,ShiftS, DoneS);
25    signal state:states := InitS;
26
27    begin
28    Done <= '1' when state = InitS or state = DoneS else '0';
29    Load <= '1' when state = LoadS else '0';
30    AddA <= '1' when state = AddS else '0';
31    Shift <= '1' when state = ShiftS else '0';
32    process(Clk)
33    begin
34       if falling_edge(Clk) then
35          case state is
36
37          --InitS waits for Start
38
39          when InitS =>
40          if Start = '1' then
41          state <= LoadS;
42          else
43          state <= InitS;
44          end if;
```

```
44       end if;
45
46    -- InitS loads the multiplier and multiplicand regs
47    --controller checks current multiplier
48    when LoadS =>
49    if Q0 = '1' then
50     state <= AddS;
51     else
52     state <= ShiftS;
53     end if;
54
55    --Adds loads adder result into accumulator
56
57    when AddS =>
58    state <= ShiftS;
59
60    --shiftS shifts the accumulator and multipler regs
61    -- ifC4 is high, the multiplier bits are processed
62    when shiftS =>
63    if C4 = '1' then
64    state <= DoneS;
65    elsif Q0 = '1' then
66    state <= AddS;
67    else
68    state <= ShiftS;
69
70    end if;
71
72    --DoneS holds done high until start goes low
73    when DoneS =>
74    if Start = '0' then
75    state <= InitS;
76    else
77    state <= DoneS;
78    end if;
79
80
81
82    l case;
83    ;
84    ocess;
85
86    havioral;
87
```

Figure 5 *Counter*

```
1
2   library IEEE;
3   use IEEE.STD_LOGIC_1164.ALL;
4   use IEEE.NUMERIC_STD.ALL;
5
6
7
8   entity CounterN is
9     generic (N : integer :=3);
10    port ( Clk : in  STD_LOGIC;
11             Clear : in  STD_LOGIC;
12             Enable : in  STD_LOGIC;
13             C4 : out  STD_LOGIC);
14  end CounterN;
15
16  architecture Behavioral of CounterN is
17
18  --internal counter sig
19  --N=3 gives a 3-bit counter, can count up to 4
20  signal Count : unsigned(N-1 downto 0) := (others => '0');
21
22
23  begin
24
25  process(Clk)
26  begin
27  if rising_edge(Clk)  then
28
29  -- clear the counter at start of multiplication
30  if Clear = '1' then
31  Count <= (others => '0');
32
33  elsif Enable = '1' then
34  Count <= Count +1;
35  end if;
36
37  end if;
38
39  end process;
40
41  -- c4 becomes high afyer 4 shift cycles
42  C4<= '1' when Count = to_unsigned(4,N) else '0';
43
44  end Behavioral;
```

Figure 6 *Adder*

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3   use IEEE.NUMERIC_STD.ALL;
4
5   entity AdderN is
6      generic (N: integer := 4);
7      port (   A: in std_logic_vector(N-1 downto 0);
8               B: in std_logic_vector(N-1 downto 0);
9               S: out std_logic_vector(N downto 0)
10           );
11  end AdderN;
12
13  architecture Behavioral of AdderN is
14  begin
15     S <= std_logic_vector(('0' & UNSIGNED(A)) + UNSIGNED(B));
16  end Behavioral;
17
```

Figure 7 *Register*

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RegN is
   generic (N: integer := 4);
   port (   Din: in std_logic_vector(N-1 downto 0);
            Dout: out std_logic_vector(N-1 downto 0);
            Clk: in std_logic;
            Load: in std_logic;
            Shift: in std_logic;
            Clear: in std_logic;
            SerIn: in std_logic
        );
end RegN;

architecture Behavioral of RegN is
   signal Dinternal: std_logic_vector(N-1 downto 0);
begin
   process(Clk)
   begin
      if (rising_edge(Clk)) then
         if (Clear = '1') then
            Dinternal <= (others => '0');
         elsif (Load = '1') then
            Dinternal <= Din;
         elsif (Shift = '1') then
            Dinternal <= SerIn & Dinternal(N-1 downto 1);
         end if;
      end if;
   end process;
   Dout <= Dinternal;
end Behavioral;
```

The system controller-based architecture is shown in Figure 1. There are two 4-bit shift registers (74LS194), a custom 9-bit shift register, a 4-bit counter (74LS163), a 4-bit parallel adder, and a system controller. The custom register operates exactly like 74LS194 except it holds 9 bits instead of 4 bits. Their control inputs S_1 and S_0 operate as: $S_1S_0 = \{00, 01, 10, 11\} = \{Hold, Shift\ Right, Shift\ Left, Load\}$. The 4-bit parallel adder takes two 4-bit numbers $(X_4X_3X_2X_1)$ and $(Y_4Y_3Y_2Y_1)$ along with an initial carry C_1 and produces their sum $(S_4S_3S_2S_1)$ and a new carry C_5.

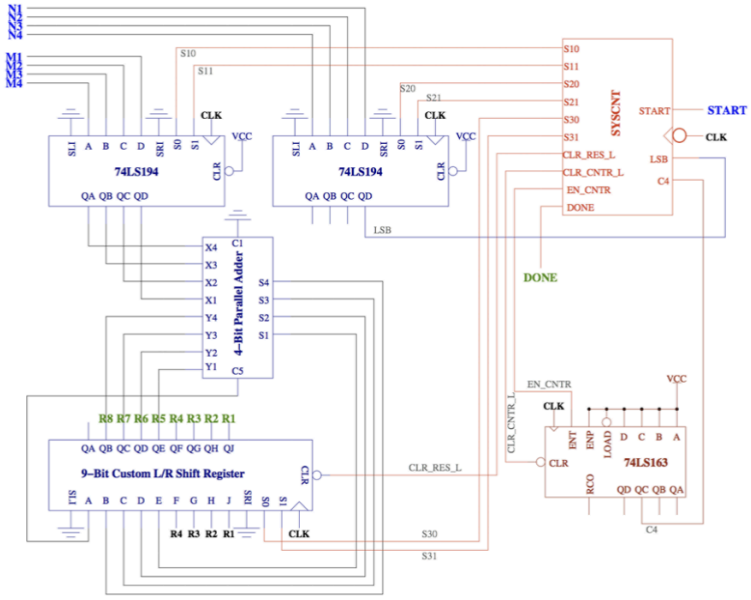

Figure 1: System controller-based architecture for 4-bit binary multiplier showing three shift registers, one counter, one parallel adder, and one system controller.

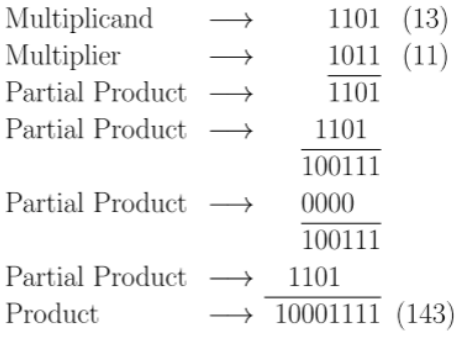

Figure 2: Overall binary multiplication process.

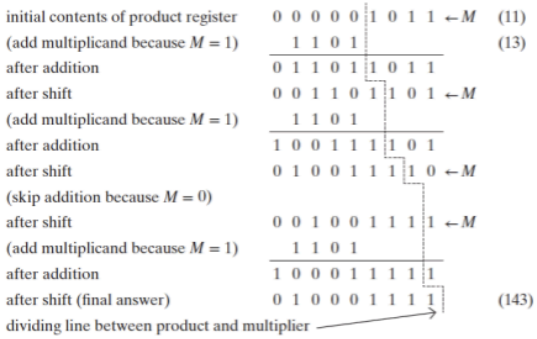

Figure 3: Detailed binary multiplication process showing contents of each shift register in the system controller-based architecture.

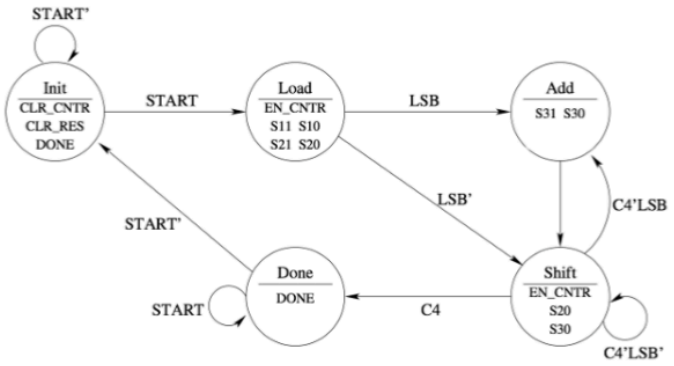

Figure 4: State diagram for the system controller in the 4-bit binary multiplier circuit.